

Aprendizaje Automático y Redes Neuronales

Ing. Laura Aquili

Temas

1. Introducción al Aprendizaje Automático.....	4
2. ¿Qué es el Aprendizaje Automático?	4
3. Aprendizaje automático como una función	5
4. Funcionamiento del Aprendizaje Automático.....	7
5. Tipos de aprendizaje automático	8
5.1. Aprendizaje automático supervisado	8
5.1.1. Regresión	8
5.1.2. Clasificación	9
5.2. Aprendizaje automático no supervisado.....	9
5.2.1. Clustering.....	10
6. Aprendizaje Profundo.....	10
7. Modelos Conexionistas.....	11
7.1. Origen del paradigma de computación conexionista.....	11
8. Redes Neuronales Biológicas.....	12
9. Redes Neuronales Artificiales	13
9.1. Definición de Red Neuronal.....	13
9.2. Estructura de las Redes Neuronales	14
9.3. Comparación entre Red Neuronal Biológica y Red Neuronal Artificial	18
9.4. Formas de interconexión de las Redes Neuronales Artificiales (RNA)	18
9.5. Características de las Redes Neuronales Artificiales	19
10. Ventajas y Desventajas de las Redes Neuronales artificiales	20
10.1. Ventajas que ofrecen las RNA	20
10.2. Desventajas que ofrecen las RNA.....	22
11. Mecanismos de Aprendizaje	23
11.1. Aprendizaje Supervisado	24
Aprendizaje por corrección de error	24
Aprendizaje por refuerzo.....	24
Aprendizaje estocástico.....	25
11.2. Aprendizaje No Supervisado	25
Aprendizaje hebbiano.....	26
Aprendizaje competitivo y comparativo	27
12. El Perceptrón	27
12.1 Aprendizaje del perceptrón.....	29
12.2 La separación lineal y el problema del XOR.....	31

13. Redes de Hopfield.....32

14. Máquinas de Boltzman35

1. Introducción al Aprendizaje Automático

El aprendizaje automático es la base de la mayoría de las soluciones de inteligencia artificial. Desde los años 50, los investigadores, a menudo, conocidos como científicos de datos, han trabajado en diferentes enfoques relacionados con la inteligencia artificial. La mayoría de las aplicaciones modernas de IA tienen sus orígenes en el aprendizaje automático, una rama de la inteligencia artificial que combina la informática y las matemáticas.

El aprendizaje automático es de muchas maneras la intersección de dos materias: la ciencia de datos y la ingeniería de software. El objetivo del aprendizaje automático es usar datos para crear un modelo predictivo que se pueda incorporar a una aplicación o servicio de software. Para lograr este objetivo, es necesaria la colaboración entre científicos de datos que exploran y preparan los datos antes de usarlos para entrenar un modelo de Machine Learning y los desarrolladores de software que integran los modelos en aplicaciones en las que se usan para predecir nuevos valores de datos (un proceso conocido como inferencia).

En esencia, el aprendizaje automático se refiere a la capacidad de un sistema informático para aprender de los datos sin programarlo explícitamente. Un ejemplo es el filtrado de spam en los correos electrónicos. Al detectar patrones similares en los mensajes de spam, las plataformas de correo electrónico pueden saber qué mensajes son útiles y cuáles deben quedar fuera de la bandeja de entrada.

Pero, ¿cómo opera el aprendizaje automático? El punto de partida son los datos. Montones de datos. Los algoritmos de aprendizaje automático se entrenan con enormes conjuntos de datos que aprenden a analizar para identificar patrones, relaciones y tendencias. En una etapa posterior, estos patrones pueden utilizarse para hacer predicciones o tomar decisiones acerca de nuevos datos no vistos anteriormente.

El aprendizaje automático brinda notables ventajas y aplicaciones en nuestra vida cotidiana.

Para empezar, se examinará un ejemplo real de cómo se puede usar el aprendizaje automático para solucionar un problema difícil.

Las técnicas agrícolas sostenibles son esenciales para maximizar la producción alimentaria y proteger un medioambiente frágil. *The Yield*, una empresa de tecnología agrícola australiana, usa sensores, datos y aprendizaje automático para ayudar a los agricultores a tomar decisiones fundamentadas, relacionadas con las condiciones meteorológicas, del suelo y los cultivos.

"Cualquier agricultor estaría de acuerdo en que cada cosecha es diferente. Podemos usar la IA para predecir la luz, el viento y la lluvia. Esto ayuda a los agricultores a saber cuándo sembrar, regar y cosechar".

2. ¿Qué es el Aprendizaje Automático?

El aprendizaje automático tiene sus orígenes en las estadísticas y el modelado matemático de los datos. La idea fundamental del aprendizaje automático es usar datos de observaciones anteriores para predecir resultados o valores desconocidos. Por ejemplo:

- El propietario de una tienda de helados podría usar una aplicación que combina las ventas históricas y los registros meteorológicos para predecir cuántos helados es probable que vendan en un día determinado, en función de la previsión meteorológica.
- Un médico podría usar datos clínicos de pacientes anteriores para ejecutar pruebas automatizadas que predicen si un nuevo paciente está en riesgo de diabetes en función de factores como el peso, el nivel de glucosa en sangre y otras medidas.
- Un investigador en la Antártida podría utilizar observaciones pasadas para automatizar la identificación de diferentes especies de pingüinos (como Emperador, *Adelia*, *Juanito* o *Barbijo*) basándose en las medidas de las aletas, el pico y otros atributos físicos del ave.

El aprendizaje automático (AA) es un tipo (subárea) de inteligencia artificial que **permite a las máquinas aprender de los datos** sin necesidad de programarlas explícitamente. Para ello, optimiza los parámetros del modelo (es decir, variables internas) mediante cálculos, de manera que el comportamiento del modelo refleje los datos o la experiencia. Como paso siguiente, el algoritmo de aprendizaje actualiza continuamente los valores de los parámetros a medida que el aprendizaje progresa, lo que permite al modelo de AA aprender y hacer predicciones o tomar decisiones basadas en la ciencia de datos.

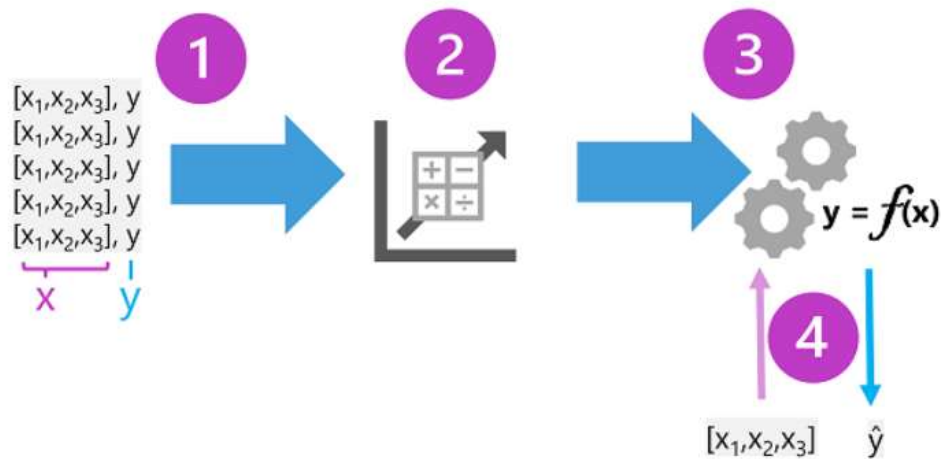
Las aplicaciones del aprendizaje automático son muy amplias y abarcan sectores como la salud, las finanzas, el marketing, el transporte, etc. Los modelos de aprendizaje automático ya se utilizan para el reconocimiento de imágenes, el procesamiento del lenguaje natural, la detección de fraudes, los sistemas de recomendación, los vehículos autónomos y la medicina personalizada.

Globalmente, el aprendizaje automático desempeña un papel crucial ya que permite a las computadoras aprender de las experiencias y los datos para mejorar su desempeño en tareas específicas sin necesidad de programarlas. Alberga el potencial de revolucionar diversas industrias al automatizar procesos complejos y **realizar predicciones o tomar decisiones inteligentes** al «digerir» grandes cantidades de información.

3. Aprendizaje automático como una función

Dado que el aprendizaje automático se basa en matemáticas y estadísticas, es habitual pensar en modelos de aprendizaje automático en términos matemáticos. Fundamentalmente, un modelo de aprendizaje automático es una aplicación de software que encapsula una *función* para calcular un valor de salida basado en uno o varios valores de entrada. El proceso de definición de esa función se conoce como *entrenamiento*. Una vez definida la función, puede usarla para predecir nuevos valores en un proceso denominado *inferencia*.

Vamos a explorar los pasos implicados en el entrenamiento y la inferencia:



1. Los datos de entrenamiento constan de observaciones pasadas. En la mayoría de los casos, las observaciones incluyen los atributos observados o *características* de la cosa observada, y el valor conocido de la cosa para la que se quiere entrenar un modelo de predicción (conocido como la *etiqueta*).

En términos matemáticos, a menudo verá que se hace referencia a las características usando el nombre abreviado de variable $\mathbf{x}$, y que se hace referencia al etiquetado como $\mathbf{y}$. Normalmente, una observación consta de múltiples valores de características, por lo que $\mathbf{x}$ es en realidad un *vector* (una matriz con múltiples valores), como esto $[\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots]$.

Para que esto quede más claro, veamos los ejemplos descritos anteriormente:

- En el escenario de ventas de helados, nuestro objetivo es entrenar un modelo que pueda predecir el número de ventas de helados en función del tiempo. Las medidas meteorológicas del día (temperatura, lluvia, velocidad del viento, etc.) serían las *características* ($\mathbf{x}$) y el número de helados vendidos en cada día sería la *etiqueta* ($\mathbf{y}$).
 - En el escenario médico, el objetivo es predecir si un paciente está o no en riesgo de diabetes en función de sus mediciones clínicas. Las medidas del paciente (peso, nivel de glucosa en sangre, etc.) son las *características* ($\mathbf{x}$) y la probabilidad de diabetes (por ejemplo, 1 para riesgo, 0 para no riesgo) es la *etiqueta* ($\mathbf{y}$).
 - En el escenario de la investigación en la Antártida, queremos predecir la especie de un pingüino basándonos en sus atributos físicos. Las medidas clave del pingüino (longitud de sus aletas, anchura de su pico, etc.) son las *características* ($\mathbf{x}$), y la especie (por ejemplo, 0 para Emperador, 1 para Adelia, 2 para Juanito, o 3 para Barbijo) es la *etiqueta* ($\mathbf{y}$).
2. Se aplica un *algoritmo* a los datos para intentar determinar una relación entre las características y el etiquetado, y generalizar esa relación como un cálculo que puede realizarse sobre $\mathbf{x}$ para calcular $\mathbf{y}$. El algoritmo específico utilizado depende del tipo de problema de predicción que esté intentando resolver, pero el principio básico es intentar *ajustar* una función a los datos, en los que los valores de las características puedan usarse para calcular la etiqueta.
 3. El resultado del algoritmo es un *modelo* que encapsula el cálculo derivado por el algoritmo como una *función*, llamémosla f . En notación matemática:

$$\mathbf{y} = f(\mathbf{x})$$

4. Ahora que se ha completado la fase de *entrenamiento*, se puede usar el modelo entrenado para la *inferencia*. El modelo es básicamente un programa de software que encapsula la función generada por el proceso de entrenamiento. Puede introducir un conjunto de valores de características y recibir como salida una predicción de la etiqueta correspondiente. Dado que la salida del modelo es una predicción calculada por la función y no un valor observado, a menudo verá la salida de la función que se muestra como $\hat{y}$.

4. Funcionamiento del Aprendizaje Automático

Entonces, ¿cómo aprenden las máquinas?

La respuesta es: a partir de los datos. En el mundo actual, se crean enormes volúmenes de datos en el transcurso de la vida cotidiana. A partir de los mensajes de texto, los correos electrónicos y las publicaciones en redes sociales a las que se envían las fotografías y vídeos que se toman en los teléfonos, se generan cantidades masivas de información. Millones de sensores crean todavía más datos en hogares, automóviles, ciudades, infraestructura de transporte público y fábricas.

Los científicos de datos pueden usar todos esos datos para entrenar modelos de Machine Learning que pueden realizar predicciones e inferencias en función de las relaciones que encuentran en los datos.

Los modelos de Machine Learning intentan capturar la relación entre los datos.

El primer paso del aprendizaje automático es recoger datos relevantes que pueden proceder de fuentes tales como bases de datos, sensores o Internet.

- **Recopilación de datos:** Se recolecta una gran cantidad de datos relevantes para el problema que se quiere resolver.
- **Preparación de los datos:** una vez recogidos los datos, es necesario preprocesarlos para garantizar su calidad e idoneidad para el análisis. Los datos se limpian, transforman y se dividen en conjuntos de entrenamiento y prueba.
- **Selección del algoritmo:** Se elige un algoritmo de aprendizaje automático adecuado según el tipo de problema (clasificación, regresión, clustering, etc.).
- **Entrenamiento del modelo:** El algoritmo se entrena con los datos de entrenamiento para encontrar patrones y relaciones. Aprende a hacer predicciones o tomar decisiones a partir de los datos de entrada.
- **Evaluación del modelo:** una vez entrenado un modelo, es necesario evaluarlo para valorar su desempeño y determinar si satisface los criterios deseados. Se evalúa el rendimiento del modelo con los datos de prueba para verificar su precisión.
- **Implementación y supervisión:** tras un entrenamiento y una evaluación satisfactorios, el modelo puede implementarse en aplicaciones de aprendizaje automático del mundo real.

Por ejemplo, imagine que una organización de conservación del medioambiente busca voluntarios para identificar y catalogar diferentes especies de flores silvestres mediante una aplicación telefónica.

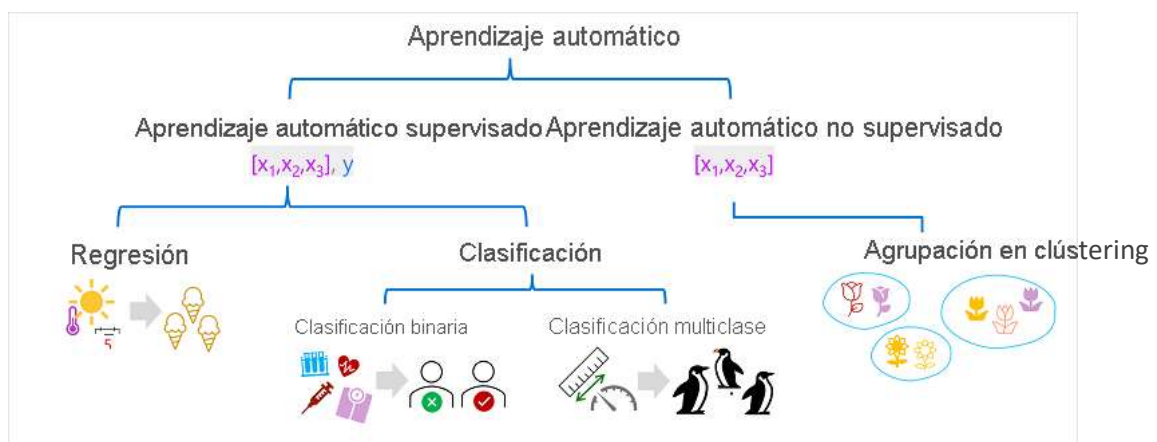
¿Cómo se puede usar el aprendizaje automático para habilitar este escenario?

1. Un equipo de botánicos y científicos recopila datos sobre muestras de flores silvestres.

2. El equipo etiqueta las muestras con la especie correcta.
3. Los datos etiquetados se procesan con un algoritmo que encuentra relaciones entre las características de las muestras y las especies etiquetadas.
4. Los resultados del algoritmo se encapsulan en un modelo.
5. Cuando los voluntarios encuentran muestras nuevas, el modelo puede identificar la etiqueta de la especie correcta.

5. Tipos de aprendizaje automático

Hay varios tipos de aprendizaje automático y debe aplicar el tipo adecuado en función de lo que esté intentando predecir. En el diagrama siguiente se muestra un desglose de los tipos comunes de aprendizaje automático.



5.1. Aprendizaje automático supervisado

El aprendizaje automático *supervisado* es un término general para los algoritmos de aprendizaje automático en los que los datos de entrenamiento incluyen valores de *características* y valores de *etiqueta* conocidos. El aprendizaje automático supervisado se utiliza para entrenar modelos determinando una relación entre las características y las etiquetas en observaciones pasadas, de modo que puedan predecirse etiquetas desconocidas para las características en casos futuros.

5.1.1. Regresión

La *regresión* es una forma de aprendizaje automático supervisado en la que la etiqueta inferida por el modelo es un valor numérico. Por ejemplo:

- El número de helados vendidos en un día determinado, en función de la temperatura, la precipitación y la velocidad del viento.
- El precio de venta de una propiedad en función de su tamaño en metros cuadrados, el número de habitaciones que contiene y los parámetros socioeconómicos de su ubicación.
- La eficiencia de combustible (en kilómetros por litro) de un coche en función del tamaño de su motor, peso, anchura, altura y longitud.

5.1.2. Clasificación

La *clasificación* es una forma de aprendizaje automático supervisado en el que la etiqueta representa una categorización o *clase*. Hay dos escenarios de clasificación comunes.

5.1.2.1. Clasificación binaria

En la *clasificación binaria*, la etiqueta determina si el elemento observado *es* (o *no*) una instancia de una clase específica. O bien, los modelos de clasificación binaria predicen uno de dos resultados mutuamente excluyentes. Por ejemplo:

- Si un paciente corre riesgo de padecer diabetes en función de parámetros clínicos como el peso, la edad, el nivel de glucosa en sangre, etc.
- Si un cliente bancario dejará de pagar un préstamo en función de sus ingresos, historial crediticio, edad y otros factores.
- Si un cliente de la lista de distribución de correo responderá positivamente a una oferta de marketing en función de los atributos demográficos y las compras anteriores.

En todos estos ejemplos, el modelo predice de forma binaria *verdadero/falso* o *positivo/negativo* para una única clase posible.

5.1.2.2. Clasificación multiclase

La *clasificación multiclase* amplía la clasificación binaria para predecir una etiqueta que representa una de varias clases posibles. Por ejemplo,

- Las especies de pingüino (*Emperador, Adelia, Juanito o Barbijo*) se basan en sus mediciones físicas.
- El género de una película (*comedia, terror, romance, aventura o ciencia ficción*) en función de su reparto, director y presupuesto.

En la mayoría de los escenarios que implican un conjunto conocido de varias clases, la clasificación multiclase se usa para predecir etiquetas mutuamente excluyentes. Por ejemplo, un pingüino no puede ser a la vez *Emperador* y *Adelia*. Sin embargo, también hay algunos algoritmos que puede usar para entrenar modelos de clasificación *multietiqueta*, en los que puede haber más de una etiqueta válida para una sola observación. Por ejemplo, una película podría clasificarse como *ciencia ficción* y *comedia*.

5.2. Aprendizaje automático no supervisado

El aprendizaje automático *no supervisado* implica entrenar modelos mediante datos que solo constan de valores de *características* sin etiquetas conocidas. Los algoritmos de aprendizaje automático no supervisado determinan las relaciones entre las características de las observaciones de los datos de entrenamiento.

5.2.1. Clustering

La forma más común de aprendizaje automático no supervisado es la *agrupación en clústeres*. Un algoritmo de agrupación en clústeres identifica similitudes entre observaciones en función de sus características y las agrupa en clústeres discretos. Por ejemplo:

- Agrupa flores similares en función de su tamaño, número de hojas y número de pétalos.
- Identificar grupos de clientes similares en función de los atributos demográficos y el comportamiento de compra.

En cierto modo, la agrupación en clústeres es similar a la clasificación multiclase, ya que clasifica las observaciones en grupos discretos. La diferencia es que cuando se utiliza la clasificación, ya se conocen las clases a las que pertenecen las observaciones de los datos de entrenamiento; por lo tanto, el algoritmo funciona determinando la relación entre las características y la etiqueta de clasificación conocida. En la agrupación en clústeres, no hay una etiqueta de clúster previamente conocida y el algoritmo agrupa las observaciones de datos basándose exclusivamente en la similitud de las características.

En algunos casos, la agrupación en clústeres se usa para determinar el conjunto de clases que existen antes de entrenar un modelo de clasificación. Por ejemplo, puede usar la agrupación en clústeres para segmentar a sus clientes en grupos y, a continuación, analizar esos grupos para identificar y clasificar las diferentes clases de clientes (*pequeño comprador frecuente, gran comprador ocasional, etc.*). A continuación, podría utilizar sus categorizaciones para etiquetar las observaciones en sus resultados de agrupación en clústeres y utilizar los datos etiquetados para entrenar un modelo de clasificación que prediga a qué categoría de cliente podría pertenecer un nuevo cliente.

6. Aprendizaje Profundo

El aprendizaje profundo es una forma avanzada de aprendizaje automático que intenta emular el modo en que el cerebro humano aprende. La clave del aprendizaje profundo es la creación de una red neuronal artificial que simula la actividad electroquímica de las neuronas biológicas mediante el uso de funciones matemáticas.

Las redes neuronales artificiales se componen de varias capas de neuronas, básicamente definiendo una función profundamente anidada. Esta arquitectura es la razón por la que la técnica se denomina aprendizaje profundo y los modelos que producen suelen denominarse redes neuronales profundas (DNN). Puede usar redes neuronales profundas para muchos tipos de problemas de aprendizaje automático, incluida la regresión y la clasificación, así como modelos más especializados para el procesamiento de lenguaje natural y la visión artificial.

Al igual que otras técnicas de aprendizaje automático descritas en este módulo, el aprendizaje profundo implica ajustar los datos de entrenamiento a una función que puede predecir una etiqueta (y) en función del valor de una o varias características (x). La función ($f(x)$) es la capa externa de una función anidada en la que cada capa de la red neuronal encapsula funciones que operan sobre x y los valores de peso (w) asociados a ellas. El algoritmo utilizado para entrenar el modelo implica alimentar iterativamente los valores de las características (x) en los datos de entrenamiento a través de las capas para calcular los valores de salida para $\hat{y}$, validar el modelo para evaluar la distancia que

separa los valores $\hat{y}$ calculados de los valores y conocidos (lo que cuantifica el nivel de error, o pérdida, en el modelo), y luego modificar los pesos (w) para reducir la pérdida. El modelo entrenado incluye los valores de peso finales que dan como resultado las predicciones más precisas.

7. Modelos Conexionistas

El cerebro es un procesador de información con unas características muy notables: es capaz de procesar a gran velocidad grandes cantidades de información procedentes de los sentidos, combinarla o compararla con la información almacenada y dar respuestas adecuadas, incluso en situaciones nuevas. Logra discernir un susurro en una sala ruidosa, distinguir una cara en una calle mal iluminada o leer entre líneas en una declaración política; pero lo más impresionante de todo es su capacidad de aprender a representar la información necesaria para desarrollar tales habilidades sin instrucciones explícitas para ello.

Aunque todavía se ignora mucho sobre la forma en que el cerebro aprende a procesar la información, se han desarrollado modelos que tratan de mimetizar tales habilidades, denominados **Redes Neuronales Artificiales o modelos de computación conexionista**.

La elaboración de estos modelos supone en primer lugar la deducción de los rasgos o características esenciales de las neuronas y sus conexiones, y en segundo lugar la implementación del modelo en una computadora de forma que se pueda simular.

Es obvio decir que estos modelos son idealizaciones burdas de las auténticas redes neuronales, en muchos casos de dudosa plausibilidad neurofisiológica, pero que sin embargo resultan interesantes cuanto menos por sus capacidades de aprendizaje.

7.1. Origen del paradigma de computación conexionista

La Inteligencia Artificial, entendida muy ampliamente como el modelado y la simulación de las actividades cognitivas complejas (percepción, memoria, solución de problemas, etc.) que caracterizan a los organismos avanzados y en particular a los seres humanos, se separó casi desde su inicio en dos ramas bien diferenciadas:

- Por un lado, se trató de modelar la actividad racional mediante **sistemas formales de reglas y manipulación simbólica** (generalmente mediante sistemas lógicos), constituyendo quizás la rama más conocida de la IA, que podríamos denominar simbólico-deductiva (se postulan una serie de reglas y el sistema resuelve los problemas realizando deducciones sobre las reglas existentes).
- Por otro lado, se desarrollaron modelos computacionales inspirados en las redes neuronales biológicas, denominados inductivos o subsimbólicos.

Si bien es mucho más conocida la aproximación simbólico-deductiva y su principal aplicación: los sistemas expertos (sistemas o agentes basados en conocimiento), existe un considerable y renacido interés por los **modelos conexionistas**. El progreso de las neurociencias nos está conduciendo a una comprensión cada vez mayor de la estructura física y lógica del cerebro; los avances tecnológicos ofrecen recursos cada vez mayores para representar estructuras muy complejas, realizar cálculos a gran velocidad y en paralelo, apoyando y fomentando así la investigación en este campo.

Podríamos situar el origen de los modelos conexionistas con la definición de la neurona formal dada por McCulloch y Pitts en 1943 como un dispositivo binario con varias entradas salidas.

El Psicólogo, D.O. Hebb, introdujo en 1949 dos ideas fundamentales que han influido de manera decisiva en el campo de las redes neuronales:

- La idea de que una percepción o un concepto se representa en el cerebro por un conjunto de neuronas activas simultáneamente y,

- La idea de que la memoria se localiza en las conexiones entre las neuronas (sinapsis).

Las hipótesis de Hebb presentan de manera intuitiva el modo en que las neuronas memorizan información, y se plasman sintéticamente en la famosa Regla de aprendizaje de Hebb.

Esta regla indica que las conexiones entre dos neuronas se refuerzan si ambas son activadas.

8. Redes Neuronales Biológicas

A grandes rasgos, recordemos que el cerebro humano se compone de decenas de billones de neuronas interconectadas entre sí formando circuitos o redes que desarrollan funciones específicas.

Una **neurona** típica recoge señales procedentes de otras neuronas a través de una pléyade de delicadas estructuras llamadas **dendritas**. La neurona emite impulsos de actividad eléctrica a lo largo de una fibra larga y delgada denominada **axón**, que se divide en millares de ramificaciones.

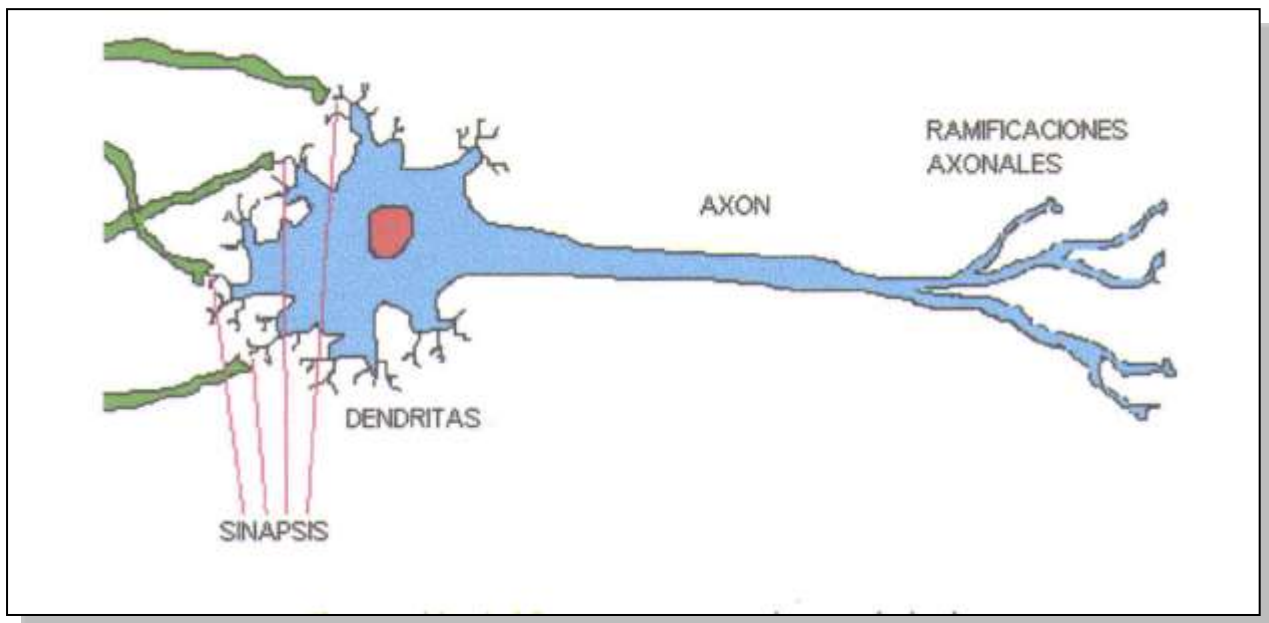


Figura 5.1

Las extremidades de estas ramificaciones llegan hasta las dendritas de otras neuronas y establecen unas conexiones llamadas **sinapsis**, en las cuales se produce una transformación del impulso eléctrico en un mensaje neuroquímico, mediante la liberación de unas sustancias llamadas **neurotransmisores**.

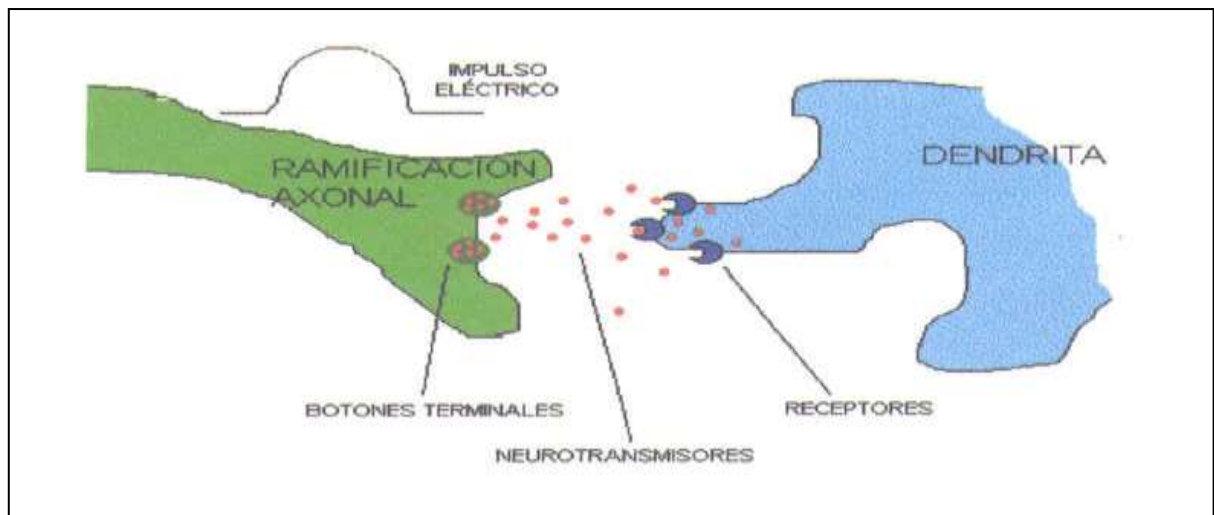


Figura 5.2

El **efecto de los neurotransmisores** sobre la neurona receptora puede ser **excitatorio o inhibitorio**, y es variable, de manera que podemos hablar de la fuerza o efectividad de una sinapsis. Las señales excitatorias e inhibitorias recibidas por una neurona se combinan, y en función de la estimulación total recibida, la neurona toma un cierto nivel de activación, que se traduce en la **generación de breves impulsos nerviosos** con una determinada frecuencia o tasa de disparo, y su propagación a lo largo del axón hacia las neuronas con las cuales sinapta.

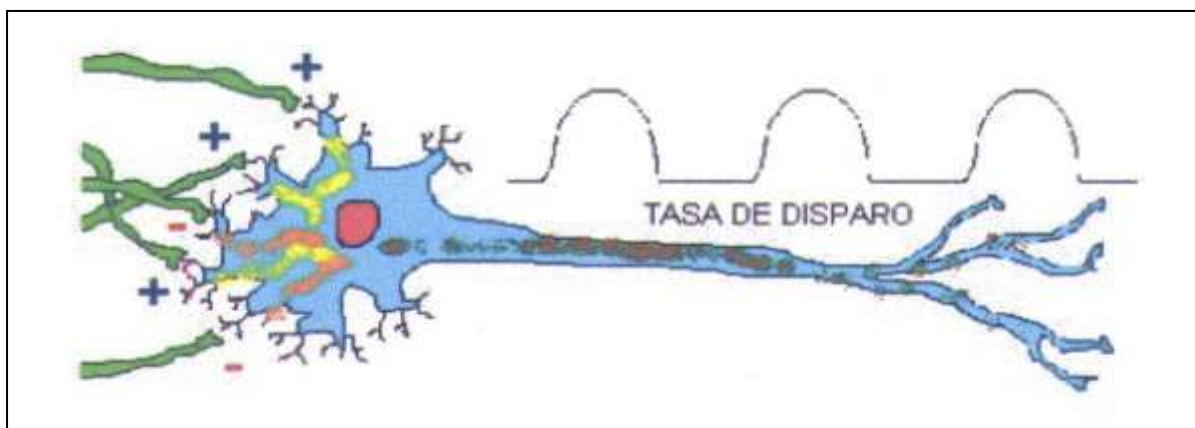


Figura 5.3

De esta manera la información se transmite de unas neuronas a otras y va siendo procesada a través de las conexiones sinápticas y las propias neuronas.

El aprendizaje de las redes neuronales se produce mediante la variación de la efectividad de las sinapsis, de esta manera cambia la influencia que unas neuronas ejercen sobre otras, de aquí se deduce que la arquitectura, el tipo y la efectividad de las conexiones en un momento dado, representan en cierto modo la memoria o estado de conocimiento de la red.

9. Redes Neuronales Artificiales

9.1. Definición de Red Neuronal

Darpa (1988), define una red neuronal como un sistema compuesto de muchos elementos simples de procesamiento los cuales operan en paralelo y cuya función es determinada por:

- La estructura de la red y
- El peso de las conexiones

realizándose el procesamiento en cada uno de los nodos o elementos de cómputo.

Según Haykin (1994), una red neuronal es un procesador paralelo masivamente distribuido que tiene una facilidad natural para el almacenamiento de conocimiento obtenido de la experiencia para luego hacerlo utilizable.

9.2. Estructura de las Redes Neuronales

Las neuronas se modelan mediante **unidades de proceso**.

Cada unidad de proceso se compone de:

- una red de conexiones de entrada
- una función de red (de propagación), encargada de computar la entrada total combinada de todas las conexiones
- un núcleo central de proceso, encargado de aplicar la función de activación
- la salida, por donde se transmite el valor de activación a otras unidades

La función de red es típicamente el sumatorio ponderado, mientras que la función de activación suele ser alguna función de umbral o una función sigmoideal.

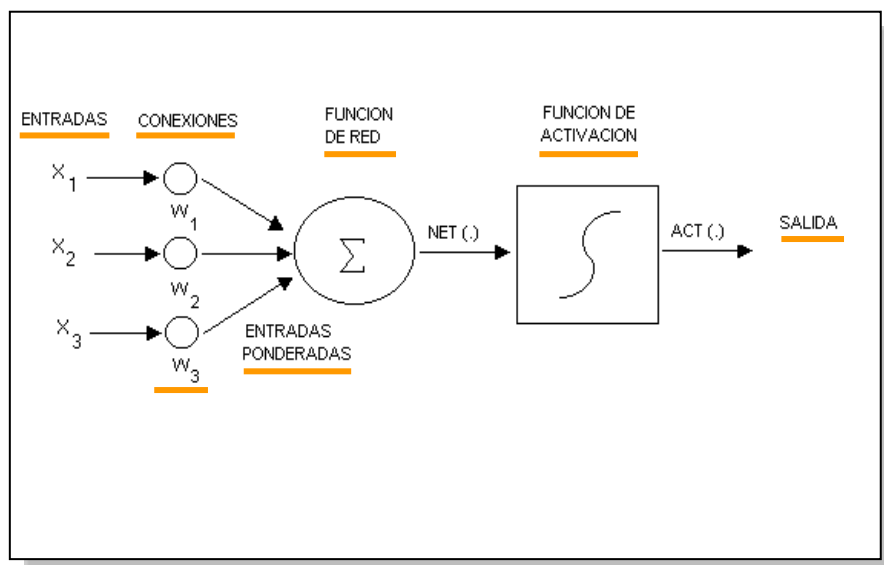


Figura 5.4

Conexiones ponderadas: hacen el papel de las conexiones sinápticas.

Peso Sináptico (w_i): el peso de la conexión equivale a la fuerza o efectividad de la sinapsis.

La existencia de conexiones determina si es posible que una unidad influya sobre otra.

El valor de los pesos y el signo de los mismos define el tipo (excitatorio/inhibitorio) y la intensidad de la influencia.

Función de Red o de Propagación: calcula el valor de base o entrada total a la unidad, generalmente como simple suma ponderada de todas las entradas recibidas, es decir de las entradas multiplicadas por el peso o valor de las conexiones.

Equivale a la combinación de las señales excitatorias e inhibitoras de las neuronas biológicas.

Función de Activación: es quizás la característica principal o definitoria de las neuronas, la que mejor define el comportamiento de las mismas.

Se encarga de calcular el nivel o estado de activación de la neurona en función de la entrada total.

Salida: calcula la salida de la neurona en función de la activación de la misma, aunque normalmente no se aplica más que la función identidad, y se toma como salida el valor de activación.

El valor de salida cumpliría la función de la tasa de disparo en las neuronas biológicas.

Función de Red o de Propagación

Como ya hemos comentado, se encarga de calcular la entrada total de la neurona como combinación de todas las entradas.

Podemos citar entre las más importantes:

- **Función lineal de base (FLB)**

Es la más utilizada. Consiste en la sumatoria ponderada de todas las entradas. Se trata de una función de tipo hiperplano, esto es, de primer orden.

Dado una unidad j , y n unidades conectadas a ésta, si llamamos X al vector de entradas (que coincide con las salidas de las unidades de la capa anterior) y W_j al vector de pesos de las conexiones correspondientes, esta función quedaría así:

$$net_j(X, W_j) = \sum_{i=1}^n x_i w_{ij}$$

Al representar los pesos utilizamos dos subíndices para indicar que conectan dos unidades, i y j , donde j se refiere a la unidad actual.

- **Función radial de base (FRB)**

Función de tipo hiperesférico, de segundo orden, no lineal. El valor de red representa la distancia a un determinado patrón de referencia.

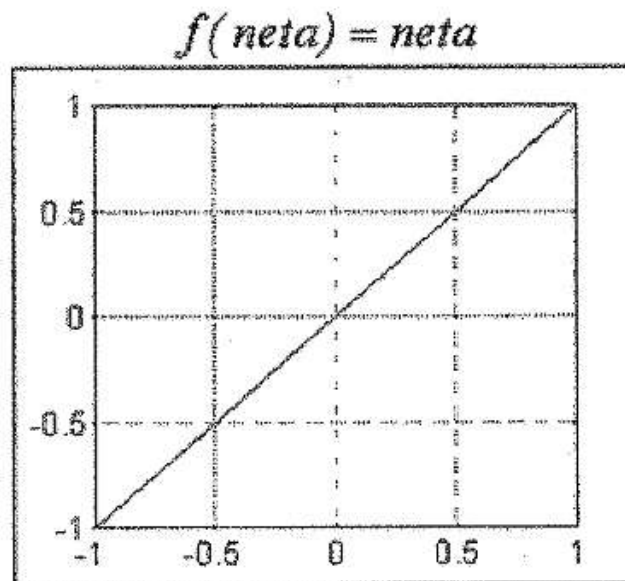
$$net_j(X, W_j) = \sqrt{\sum_{i=1}^n (x_i - w_{ij})^2}$$

Función de Activación

Podemos distinguir entre:

- **Funciones lineales**

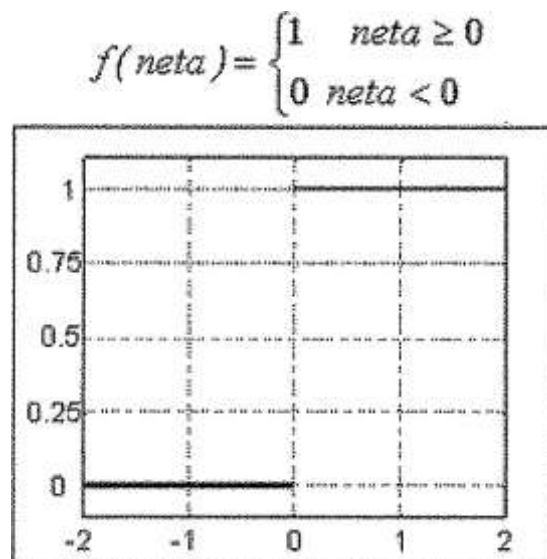
En las que la salida es proporcional a la entrada.



- Funciones de umbral

En las cuales la salida es un valor discreto (típicamente binario 0 o 1) que depende de si la estimulación total supera o no un determinado valor de umbral.

En un principio se pensó que las neuronas usaban una función de umbral, es decir, que permanecían inactivas y se activaban solo si la estimulación total superaba cierto valor límite. Esto se puede modelar con una función escalón: la más típica es el escalón unitario: la función devuelve 0, por debajo del valor crítico (umbral) y 1, por encima.



Después se comprobó que las neuronas emitían impulsos de actividad eléctrica con una frecuencia variable, dependiendo de la intensidad de la estimulación recibida, y que tenían cierta actividad hasta en reposo, con estimulación nula. Estos descubrimientos llevaron al uso de funciones no lineales con esas características, como la función sigmoideal, con un perfil parecido al escalón de una función de umbral, pero continua.

- **Funciones no lineales**

No proporcionales a la entrada.

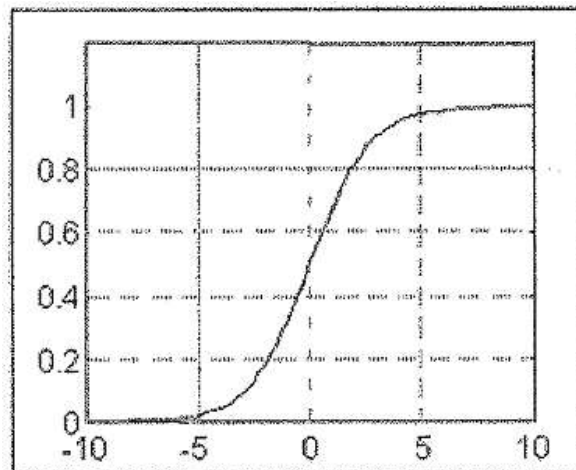
Función Sigmoidal o logística

Se trata de una función continua no lineal que posee un rango comprendido entre 0 y 1. Esto, aplicado a las unidades de proceso de una red neuronal artificial significa que, sea cual sea la entrada, la salida estará comprendida entre 0 y 1. Es decir, es una función que transforma cualquier valor real (positivo o negativo) en un valor entre **0 y 1** (su salida está acotada entre 0 y 1).

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Donde e es el número de Euler (~ 2.718) y x es la entrada.

- Cuando x es negativo, la salida se acerca a **0**.
- Cuando x es positivo, la salida se acerca a **1**.
- Cuando x es cero, la salida es **0.5**.



La salida de una unidad vale 0.5 cuando la entrada es nula, esto significa que la unidad tiene cierta actividad aún en ausencia de estimulación. Al aumentar la estimulación la unidad aumenta su activación, y la disminuye si la estimulación es inhibitoria, de forma parecida a como se comportan las neuronas reales.

Tiene como desventaja que sufre de *desvanecimiento del gradiente*, lo que significa que para valores extremos (muy grandes o muy pequeños) la derivada es casi cero y esto dificulta el entrenamiento.

Función ReLU (Unidad Lineal Rectificada)

Es una función de activación muy usada en redes neuronales porque es simple, rápida de calcular, y da buenos resultados en la práctica. Su nombre viene de su forma matemática, que *rectifica* los valores negativos a cero y deja pasar los positivos de forma lineal.

$$\text{ReLU}(x) = \max(0, x)$$

Es decir:

- Si el valor de entrada x es **menor que 0**, la salida es **0**.
- Si x es **mayor o igual a 0**, la salida es el **mismo valor** x .

Ejemplo:

$\text{ReLU}(3.5) \rightarrow 3.5$

$\text{ReLU}(-2.1) \rightarrow 0$

$\text{ReLU}(0) \rightarrow 0$

Tiene como ventaja que ayuda a evitar el problema del **desvanecimiento del gradiente** que afectaba a funciones como la sigmoideal y hace que los modelos converjan más rápido durante el entrenamiento.

Casi todos los avances recientes en conexionismo se atribuyen a arquitecturas multicapa que utilizan funciones de activación no lineales, las cuales resultan esenciales porque permiten que las redes aprendan patrones complejos y no simplemente relaciones lineales.

9.3. Comparación entre Red Neuronal Biológica y Red Neuronal Artificial

Redes Neuronales Biológicas	Redes Neuronales Artificiales
Neuronas	Unidades de proceso
Conexiones sinápticas	Conexiones ponderadas
Efectividad de las sinapsis	Peso de las conexiones
Efecto excitatorio o inhibitorio de una conexión	Signo del peso de una conexión
Efecto combinado de las sinapsis	Función de propagación o de red
Activación $\rightarrow$ tasa de disparo	Función de activación $\rightarrow$ salida

9.4. Formas de interconexión de las Redes Neuronales Artificiales (RNA)

Para diseñar una red debemos establecer como estarán conectadas unas unidades con otras y determinar adecuadamente los pesos de las conexiones.

Lo más usual es disponer las unidades en forma de capas, pudiéndose hablar de redes de una, de dos o de más de dos capas, las llamadas redes multicapa.

Aunque inicialmente se desarrollaron redes de una sola capa, lo más usual es disponer tres o más capas:

- Capa de entrada: es la primera capa y actúa como buffer de entrada, almacenando la información bruta suministrada a la red o realizando un sencillo pre-proceso de la misma.
- Capa de salida: actúa como interfaz o buffer de salida, almacenando la respuesta de la red para que pueda ser leída.

- Capas ocultas: son las capas intermedias, principales encargadas de extraer, procesar y memorizar la información.

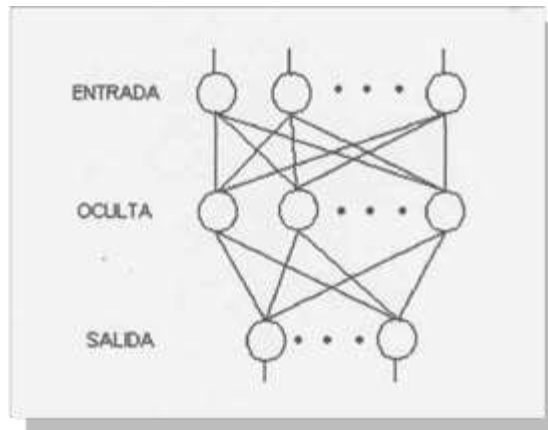


Figura 5.5

Podemos clasificar las RNA, además de por el número de capas de una red, en función de cómo se interconectan unas capas con otras:

- Redes en cascada: feed-forward. La información fluye unidireccionalmente de una capa a otra (desde la capa de entrada a las capas ocultas y de éstas a la capa de salida), y además, no se admiten conexiones intracapa.
- Redes recurrentes: feed-back. La información puede volver a lugares por los que ya había pasado, formando bucles, y se admiten las conexiones intracapa (laterales), incluso de una unidad consigo misma.

Las conexiones entre una capa y otra pueden ser:

- Totales: cada unidad se conecta con todas las unidades de la capa siguiente.
- Parciales: una unidad se conecta con solo algunas de las unidades de la capa siguiente, generalmente siguiendo algún patrón aleatorio o pseudo-aleatorio (por ejemplo, mediante algoritmos genéticos).

Desde una aproximación temporal se puede distinguir entre:

- Conexiones sin retardo
- Conexiones con retardo

9.5. Características de las Redes Neuronales Artificiales

1. Aprendizaje inductivo: no se le indican las reglas para dar una solución, sino que extrae sus propias reglas a partir de los ejemplos de aprendizaje, modifican su comportamiento en función de la experiencia. Esas reglas quedan almacenadas en las conexiones y no representadas explícitamente como en los sistemas basados en conocimiento (simbólico-deductivos).
2. Generalización: una vez entrenada, se le pueden presentar a la red datos distintos a los usados durante el aprendizaje. La respuesta obtenida dependerá del parecido de los datos con los ejemplos de entrenamiento.
3. Abstracción o tolerancia al ruido: las redes neuronales artificiales son capaces de extraer o abstraer las características esenciales de las entradas aprendidas, de esta manera pueden procesar correctamente datos incompletos o distorsionados.

4. Procesamiento paralelo: las neuronas reales trabajan en paralelo; en el caso de las redes neuronales artificiales, hay un paralelismo inherente, la estructura y modo de operación de las mismas las hace especialmente adecuadas para el procesamiento paralelo real mediante multiprocesadores. El procesamiento paralelo consiste en realizar múltiples operaciones al mismo tiempo, usando varios núcleos de CPU, GPU, TPUs, o clústeres de computadoras. Esto es clave para trabajar con tareas pesadas como el entrenamiento de redes neuronales profundas.
5. Memoria distribuida: el conocimiento acumulado por la red se halla distribuido en numerosas conexiones, esto tiene como consecuencia la tolerancia a fallos: una red neuronal es capaz de seguir funcionando adecuadamente a pesar de sufrir lesiones con destrucción de neuronas o sus conexiones, ya que la información se halla distribuida por toda la red, sin embargo en un programa tradicional un pequeño fallo en cualquier punto puede invalidarlo todo y dar un resultado absurdo o no dar ningún resultado.

10. Ventajas y Desventajas de las Redes Neuronales artificiales

10.1. Ventajas que ofrecen las RNA

1. Aprendizaje adaptativo: capacidad de aprender a realizar tareas basadas en un entrenamiento o en una experiencia inicial.
2. Auto-organización: una red neuronal puede crear su propia organización o representación de la información que recibe mediante una etapa de aprendizaje.
3. Tolerancia a fallos: la destrucción parcial de una red conduce a una degradación de su estructura, sin embargo, algunas capacidades de la red se pueden retener, incluso sufriendo un gran daño.
4. Operación en tiempo real: los cálculos de las redes neuronales pueden ejecutarse en paralelo, lo que permite obtener respuestas rápidas (minimizar la latencia en la generación de resultados). Para lograr esto, se utilizan arquitecturas de hardware especializadas, como unidades de procesamiento gráfico (GPU), unidades de procesamiento tensorial (TPU) y circuitos integrados (microchips) de aplicación específica (ASIC), que están optimizados para procesar grandes volúmenes de datos con baja latencia.
5. Fácil inserción dentro de la tecnología existente: Las redes neuronales pueden integrarse fácilmente en sistemas tecnológicos actuales gracias a la disponibilidad de **chips especializados** (como **TPUs** o **ASICs**), que están optimizados para realizar cálculos de redes neuronales de manera más rápida y eficiente.
6. Escalabilidad y Eficiencia Computacional: Ser escalable en RNA es la capacidad del modelo para crecer, adaptarse o reducirse eficientemente según los recursos disponibles, el tamaño de los datos o la complejidad del problema.

Aprendizaje adaptativo

La capacidad de aprendizaje adaptativo es una de las características más atractivas de las redes neuronales. Esto es, aprenden a llevar a cabo ciertas tareas mediante un entrenamiento con ejemplos ilustrativos.

Como las redes neuronales pueden aprender a diferenciar patrones mediante ejemplos y entrenamientos, no es necesario elaborar modelos a priori, ni especificar funciones de distribución de probabilidad.

Las redes neuronales son sistemas dinámicos autoadaptativos.

- Son adaptables debido a la capacidad de autoajuste de los elementos procesales (neuronas) que componen el sistema.
- Son dinámicos, pues son capaces de estar constantemente cambiando para adaptarse a las nuevas condiciones.

Las RNA **aprenden de los datos**. A medida que reciben más ejemplos de entrenamiento, pueden **ajustar sus pesos** y mejorar su rendimiento. Es decir, que, en el proceso de aprendizaje, los enlaces ponderados de las neuronas se ajustan de manera que se obtengan ciertos resultados específicos.

Una red neuronal no necesita un algoritmo para resolver un problema, ya que ella puede generar su propia distribución de pesos en los enlaces mediante el aprendizaje. También existen redes que continúan aprendiendo a lo largo de su vida, después de completado su período de entrenamiento.

La función del diseñador es únicamente la obtención de la arquitectura apropiada. No es problema del diseñador el como la red aprenderá a discriminar.

Sin embargo, si es necesario que desarrolle un buen algoritmo de aprendizaje que le proporcione a la red la capacidad de discriminar, mediante un entrenamiento con patrones.

Auto-organización

Las redes neuronales emplean su capacidad de aprendizaje adaptativo para autoorganizar la información que reciben durante el aprendizaje y/o la operación.

Mientras que el aprendizaje es la modificación de cada elemento procesal, la autoorganización consiste en la modificación de la red neuronal completa para llevar a cabo un objetivo específico.

Cuando las redes neuronales se usan para reconocer ciertas clases de patrones, ellas autoorganizan la información usada. Por ejemplo, la red llamada backpropagation, creará su propia representación característica, mediante la cual puede reconocer ciertos patrones.

Esta autoorganización provoca la **generalización**: facultad de las redes neuronales de responder apropiadamente cuando se les presentan datos o situaciones a las que no había sido expuesta anteriormente.

El sistema puede generalizar la entrada para obtener una respuesta. Esta característica es muy importante cuando se tiene que solucionar problemas en los cuales la información de entrada no es muy clara. Además permite que el sistema de una solución, incluso cuando la información de entrada está especificada de forma incompleta.

Tolerancia a fallos

Las redes neuronales fueron los primeros métodos computacionales con la capacidad inherente de tolerancia a fallos. Comparados con los sistemas computacionales tradicionales, los cuales pierden su funcionalidad cuando sufren un pequeño error de memoria.

En las redes neuronales, si se produce un fallo en un número no muy grande de neuronas y aunque el comportamiento del sistema se ve influenciado, no sufre una caída repentina.

Hay dos aspectos distintos respecto a la tolerancia a fallos:

- Las redes pueden aprender a reconocer patrones con ruido, distorsionados o incompletos. Esta es una tolerancia a fallos respecto a los datos.
- Las redes pueden seguir realizando su función (con cierta degradación) aunque se destruya parte de la red.

La razón por la que las redes neuronales son tolerantes a los fallos es que tienen su información distribuida en las conexiones entre neuronas, existiendo cierto grado de redundancia en este tipo

de almacenamiento. La mayoría de los ordenadores algorítmicos y sistemas de recuperación de datos almacenan cada pieza de información en un espacio único, localizado y direccionable. En cambio, las redes neuronales almacenan información no localizada. Por lo tanto, la mayoría de las interconexiones entre los nodos de la red tendrán sus valores en función de los estímulos recibidos, y se generará un patrón de salida que represente la información almacenada.

Operación en tiempo real

Una de las mayores prioridades, casi en la totalidad de las áreas de aplicación, es la necesidad de realizar procesos con datos de forma muy rápida.

Las redes neuronales se adaptan bien a esto debido a su implementación paralela. Para que la mayoría de las redes puedan operar en un entorno de tiempo real, la necesidad de cambio en los pesos de las conexiones o entrenamiento es mínimo.

Fácil inserción dentro de la tecnología existente

Una red individual puede ser entrenada para desarrollar una única y bien definida tarea (tareas complejas, que hagan múltiples selecciones de patrones requerirán sistemas de redes interconectadas). Con las herramientas computacionales existentes (no del tipo PC), una red puede ser rápidamente entrenada, comprobada, verificada y trasladada a una implementación hardware de bajo coste.

Por lo tanto, no se presentan dificultades para la inserción de redes neuronales en aplicaciones específicas, por ejemplo, de control, dentro de los sistemas existentes. De esta manera, las redes neuronales se pueden utilizar para mejorar sistemas en forma incremental y cada paso puede ser evaluado antes de acometer un desarrollo más amplio.

Escalabilidad y Eficiencia Computacional

Las redes neuronales artificiales son altamente **escalables**, lo que significa que pueden adaptarse a diferentes volúmenes de datos y niveles de complejidad sin perder eficiencia. Se pueden entrenar desde modelos pequeños para tareas simples hasta redes profundas (deep learning) con millones de parámetros para resolver problemas complejos. Además, su escalabilidad se ve potenciada por entornos de computación distribuida, GPU/TPU, y servicios en la nube, lo que permite entrenar modelos de gran tamaño en tiempos razonables y desplegarlos en producción con facilidad.

10.2. Desventajas que ofrecen las RNA

1. Definición de muchos parámetros

Las redes neuronales requieren la definición de múltiples parámetros, como definir la arquitectura más apropiada, funciones de activación, el número de capas ocultas, el número de nodos por capa, las interconexiones, etc. Aunque herramientas modernas han reducido esta carga, ya que automatizan parte de este proceso, no eliminan completamente la necesidad de diseño manual o ajuste fino.

2. Preparación de datos compleja

Preparar datos para una red neuronal (limpieza, normalización, codificación, balanceo, etc.) sigue siendo una parte crítica y laboriosa del proceso. Muchas veces se subestima cuánto tiempo y experiencia se necesita para hacerlo bien.

3. Requieren grandes cantidades de datos

Para obtener buenos resultados, especialmente en redes profundas, se necesita una gran cantidad de datos etiquetados, lo cual no siempre está disponible.

4. Alto costo computacional

Entrenar redes neuronales consume muchos recursos (tiempo, energía, GPU/TPU), lo que puede ser costoso, especialmente en proyectos pequeños.

5. Caja negra (falta de interpretabilidad)

Es difícil entender cómo la red llega a una decisión, es decir, no ofrece una interpretación fácil. En lugar de ser un sistema de apoyo a la decisión, la caja negra se puede convertir en el "tomador" de la decisión. Puede ocurrir que un director de riesgo deniegue un crédito solo porque lo dice la caja negra, sin que él pueda argumentar esta decisión ya que no entiende el funcionamiento de la red neuronal. Por ende, esto limita su uso en aplicaciones donde se necesita explicar el resultado (ej. medicina, finanzas).

6. Riesgo de sobreajuste (overfitting)

Si no se regulan bien, las RNA pueden memorizar los datos de entrenamiento en lugar de generalizar, lo que afecta su rendimiento en nuevos datos.

7. Dependencia del diseño del modelo

El rendimiento puede variar mucho según la arquitectura, el número de capas, la función de activación, etc. Esto requiere prueba y error, experiencia y tiempo.

8. Tiempo de entrenamiento prolongado

Entrenar modelos complejos puede tomar horas o incluso días, especialmente con grandes conjuntos de datos.

9. Sensibles a la calidad de los datos

Datos ruidosos, desequilibrados o mal etiquetados pueden afectar seriamente el rendimiento del modelo.

11. Mecanismos de Aprendizaje

El aprendizaje es el proceso por el cual una red neuronal modifica sus pesos en respuesta a una información de entrada.

Los cambios que se producen durante el mismo se reducen a la destrucción, modificación y creación de conexiones entre las neuronas. En los sistemas biológicos existe una continua destrucción y creación de conexiones. En los modelos de redes neuronales artificiales, la creación de una nueva conexión implica que el peso de la misma pasa a tener un valor distinto de cero. De la misma manera, una conexión se destruye cuando su peso pasa a ser cero.

Durante el proceso de aprendizaje, los pesos de las conexiones de la red sufren modificaciones, por lo tanto, se puede afirmar que este proceso ha terminado (la red ha aprendido) cuando los valores de los pesos permanecen estables.

Un aspecto importante respecto al aprendizaje de las redes neuronales es el conocer cómo se modifican los valores de los pesos, es decir, cuales son los criterios que se siguen para cambiar el valor asignado a las conexiones cuando se pretende que la red aprenda una nueva información.

Existen dos métodos de aprendizaje importantes que pueden distinguirse:

- Aprendizaje supervisado

- Aprendizaje no supervisado

Otro criterio que se puede utilizar para diferenciar las reglas de aprendizaje se basa en la siguiente consideración:

- Aprendizaje on-line

Si la red puede aprender durante su funcionamiento habitual.

- Aprendizaje off-line

Si el aprendizaje supone la desconexión de la red, es decir, su inhabilitación hasta que el proceso se termine.

Cuando el aprendizaje es off-line se distingue entre una fase de aprendizaje o entrenamiento y una fase de operación o funcionamiento, existiendo un conjunto de datos de entrenamiento y un conjunto de datos de test o prueba, que serán utilizados en la correspondiente fase. Además, los pesos de las conexiones permanecen fijos después que termina la etapa de entrenamiento de la red.

Debido, precisamente a su carácter estático, estos sistemas no presentan problemas de estabilidad en su funcionamiento.

11.1. Aprendizaje Supervisado

El aprendizaje supervisado se caracteriza porque el proceso de aprendizaje se realiza mediante un entrenamiento controlado por un agente externo (supervisor, maestro) que determina la respuesta que debería generar la red a partir de una entrada determinada.

El supervisor controla la salida de la red y en caso de que ésta no coincida con la deseada, se procederá a modificar los pesos de las conexiones, con el fin de conseguir que la salida obtenida se aproxime a la deseada.

En este tipo de aprendizaje se suelen considerar, a su vez, tres formas de llevarlo a cabo, que dan lugar a los siguientes aprendizajes supervisados:

- Aprendizaje por corrección de error
- Aprendizaje por refuerzo
- Aprendizaje estocástico

Aprendizaje por corrección de error

Consiste en ajustar los pesos de las conexiones de la red en función de la diferencia, es decir, en función del error cometido en la salida.

$$\text{Error cometido} = \text{Valor deseado} - \text{Valor obtenido}$$

Un ejemplo de este tipo de algoritmos lo constituye la regla de aprendizaje del Perceptrón, utilizada en el entrenamiento de la red del mismo nombre que desarrolló Rosenblatt en 1958. Esta es una regla muy simple, para cada neurona, en la capa de salida se le calcula la desviación a la salida objetivo como el error, δ , el cual luego se utiliza para cambiar los pesos sobre la conexión de la neurona precedente.

Aprendizaje por refuerzo

Se trata de un aprendizaje supervisado, más lento que el anterior, que se basa en la idea de no disponer de un ejemplo completo del comportamiento deseado, es decir, de no indicar durante el

entrenamiento exactamente la salida que se desea que proporcione la red ante una determinada entrada.

En el aprendizaje por refuerzo la función del supervisor se reduce a indicar mediante una señal de refuerzo si la salida obtenida en la red se ajusta a la deseada (éxito = +1 o fracaso = -1) y en función de ello se ajustan los pesos basándose en un mecanismo de probabilidades. Se podría decir que en este tipo de aprendizaje la función del supervisor se asemeja más a la de un crítico (que opina sobre la respuesta de la red) que a la de un maestro (que indica a la red la respuesta concreta que debe generar), como ocurriría en el caso de supervisión por corrección del error.

Aprendizaje estocástico

Consiste básicamente en realizar cambios aleatorios en los valores de los pesos de las conexiones de la red y evaluar su efecto a partir del objetivo deseado y de distribuciones de probabilidad.

En el aprendizaje estocástico se suele hacer una analogía en términos termodinámicos, asociando a la red neuronal con un sólido físico que tiene cierto estado energético. En el caso de la red, la energía de la misma representaría el grado de estabilidad de la red, de tal forma que el estado de mínima energía correspondería a una situación en la que los pesos de las conexiones consiguen que su funcionamiento sea el que más se ajusta al objetivo deseado.

Según lo anterior, el aprendizaje consistiría en realizar un cambio aleatorio de los valores de los pesos y determinar la energía de la red. Si la energía es menor después del cambio, es decir, si el comportamiento de la red se acerca al deseado, se acepta el cambio. Si, por el contrario, la energía no es menor, se aceptaría el cambio en función de una determinada y preestablecida distribución de probabilidades.

11.2. Aprendizaje No Supervisado

Las redes con aprendizaje no supervisado (también conocido como autosupervisado) no requieren influencia externa para ajustar los pesos de las conexiones entre sus neuronas. La red no recibe ninguna información por parte del entorno que le indique si la salida generada en respuesta a una determinada entrada es o no correcta.

Estas redes deben encontrar las características, regularidades, correlaciones o categorías que se puedan establecer entre los datos que se presenten en su entrada. Existen varias posibilidades en cuanto a la interpretación de la salida de estas redes, que dependen de su estructura y del algoritmo de aprendizaje empleado:

- En algunos casos, la salida representa el grado de familiaridad o similitud entre la información que se le está presentando en la entrada y las informaciones que se le han mostrado hasta entonces (en el pasado).
- En otro caso, podría realizar un establecimiento de categorías, indicando la red a la salida, a qué categoría pertenece la información presentada a la entrada, siendo la propia red quien debe encontrar las categorías apropiadas a partir de las correlaciones entre las informaciones presentadas.

En cuanto a los algoritmos de aprendizaje no supervisado, en general se suelen considerar dos tipos, que dan lugar a los siguientes aprendizajes:

- Aprendizaje hebbiano
- Aprendizaje competitivo y comparativo

Aprendizaje hebbiano

Los sistemas neuronales biológicos no nacen preprogramados con todo el conocimiento y las capacidades que llegarán a tener eventualmente.

Un proceso de aprendizaje que tiene lugar a lo largo de un período de tiempo modifica de alguna forma la red para incluir la nueva información.

La teoría básica procede de un libro de 1949 escrito por Hebb. La idea principal se expresaba en forma de suposición y es la siguiente:

Cuando un axón de la célula A está suficientemente próximo para excitar a una célula B o toma parte en su disparo de forma persistente, tiene lugar algún proceso de crecimiento o algún cambio metabólico en una de las células, o en las dos, de tal modo que la eficiencia de A, como una de las células que desencadena el disparo de B, se ve incrementada.

Para ilustrar la idea básica, consideramos el ejemplo clásico de condicionamiento, empleando el conocido experimento de Pavlov, reflejado en una idealización de tres neuronas que forman parte del proceso, como indica la Figura 5.6:

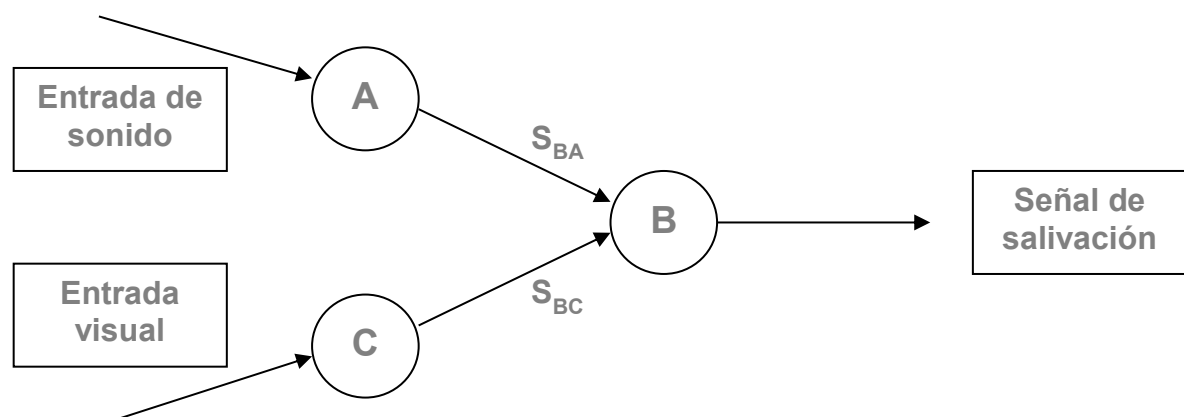


Figura 5.6

Supongamos que la excitación de C, causada por la visualización de la comida, es suficiente para excitar a B, que da lugar a la salivación. Además, supongamos que, en ausencia de estimulación adicional, la excitación de A, que se debe a oír una campanilla, no basta para dar lugar al disparo de B.

Permitamos que C de lugar a que B dispare mostrando comida al sujeto, y, mientras B sigue disparando, estimulemos a A haciendo sonar una campanilla. Dado que B sigue disparando, A participa ahora en la excitación de B, aun cuando por sí sola A no sería suficiente para dar lugar a que B disparese.

En esta situación, la suposición de Hebb determina que se produce algún cambio entre A y B, de tal modo que la influencia de A sobre B se ve incrementada. Si el experimento se repite con frecuencia suficiente, A será capaz de lograr, eventualmente, que se dispare B incluso en ausencia de la estimulación visual procedente de C.

Llevado esto al terreno de las RNA, significa que el peso de la conexión entre ambas neuronas se ve incrementado.

Aprendizaje competitivo

El aprendizaje competitivo se orienta a la clasificación de los datos de entrada. En este tipo de aprendizaje, las unidades de salida luchan por el control sobre porciones del espacio de entrada.

Las unidades de entrada se conectan directamente a las unidades de salida, pero éstas también se conectan entre sí con conexiones precableadas negativas o inhibitorias.

La unidad de salida con la mayor activación en sus entradas tendrá más fuerza que sus competidores. Como resultado, los competidores se vuelven más débiles, perdiendo su poder de inhibición sobre las unidades más fuertes. Las unidades más fuertes se hacen cada vez más fuertes y su efecto inhibitorio sobre las demás unidades de salida se hace cada vez mayor. Pronto, el resto de las unidades de salida estará completamente inactivo. Este tipo de inhibición mutua se denomina: comportamiento de el ganador se lleva todo.

12. El Perceptrón

El Perceptrón, inventado por Rosenblatt (1962), fue uno de los primeros modelos de redes neuronales.

Consiste en pesos entrenables multiplicativos, un sumador y una función umbral.

Un perceptrón imita una neurona tomando la suma ponderada de sus entradas y enviando a la salida un 1 si la suma es más grande que algún valor umbral ajustable, y enviando un 0 en caso contrario.

Un perceptrón es una representación, o sea, una red neuronal en la que:

- Solo hay una neurona
- Las entradas son binarias: solo se permiten 0 y 1
- Las cajas lógicas pueden interponerse entre las entradas y los pesos del perceptrón. Cada caja lógica puede verse como una tabla que produce un valor de salida de 0 o 1 para cada combinación de 0 y 1 que pueda aparecer en sus entradas.
- La salida del perceptrón es 0 o 1, dependiendo de si la suma ponderada de las salidas de las cajas lógicas es mayor que el umbral.

En la Figura 5.7 se puede observar un modelo de un perceptrón.

Las entradas son 0 o 1, al igual que las salidas de las cajas lógicas. Si la suma de las salidas ponderadas de las cajas lógicas es mayor que 0, la salida del perceptrón es 1 y se dice que el perceptrón dice SI, se ha reconocido una clase. En cualquier otro caso, se dice que el perceptrón dice NO, no se ha reconocido una clase.

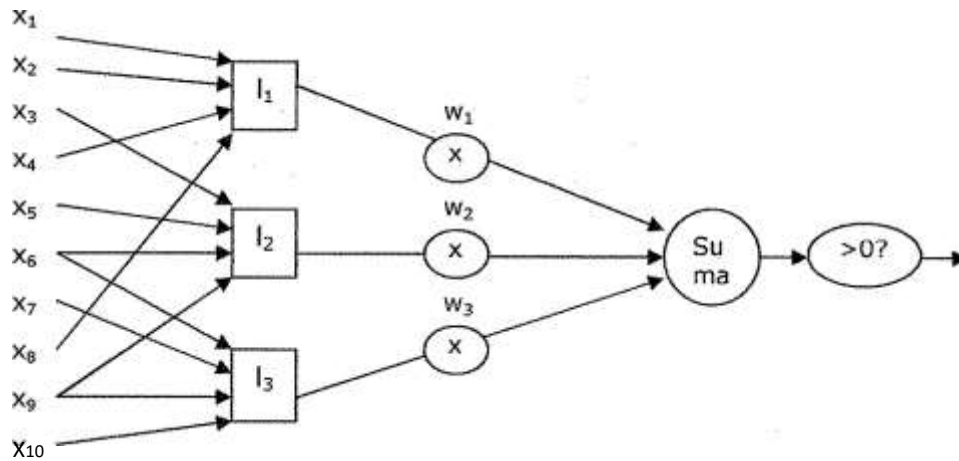


Figura 5.7

Para expresar la situación en notación matemática, suponga que la salida de la i -ésima caja lógica es l_i , el i -ésimo peso es w_i , y que finalmente, el umbral es T . Entonces la salida del perceptrón completo, P , está dada por la siguiente fórmula:

$$P = \begin{cases} 1 & \text{si } \sum_i w_i \times l_i > T \\ 0 & \text{en otro caso} \end{cases}$$

Note que, si cualesquiera de las cajas lógicas del perceptrón pudieran atender todas las entradas del perceptrón, entonces no habría necesidad de otras cajas lógicas ni tampoco de pesos o de una función de umbral. En lugar de ello, todas las combinaciones de 0 y 1 de entrada se registrarían en la tabla interna de la caja lógica, junto con la salida apropiada de 0 o 1.

Por supuesto que el número de entradas de la tabla sería de 2^n si hubiera n entradas y se tuvieran que manejar todas las posibles combinaciones de 0 y 1. Esta relación exponencial entre el número de entradas de la tabla y el número de entradas del perceptrón hace que no sea práctico el que cualesquiera de las cajas lógicas atiendan a todas las entradas, e incluso si atendiera una fracción sustancial de todas las entradas, si hubiera muchas.

En consecuencia, se supone que cada caja lógica de un perceptrón debe atender solo un pequeño número de entradas, por ejemplo, las cajas lógicas del perceptrón de la Figura 5.7 atienden un máximo de 4 entradas.

En un perceptrón limitado por el orden, de orden n , cada caja lógica atiende n o menos entradas.

Otra limitación más consiste en reducir las cajas lógicas a una sola entrada, que se pasa por el correspondiente multiplicador:

En un perceptrón directo, cada caja lógica tiene solo una entrada, y la salida es siempre la misma que la entrada.

De manera alternativa, un perceptrón directo puede verse como un perceptrón sin cajas lógicas, como se ilustra en la Figura 5.8.

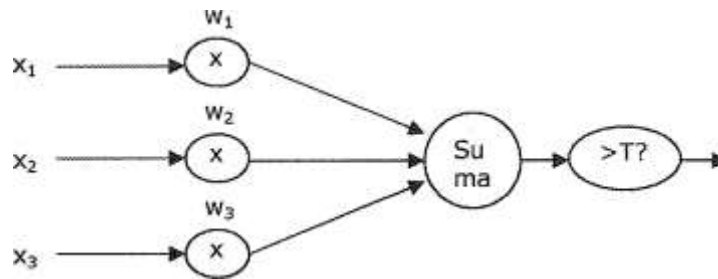


Figura 5.8

12.1 Aprendizaje del perceptrón

Es importante hacer notar que existe un procedimiento que descubre un buen conjunto de pesos para un perceptrón, dado que tal conjunto exista.

Además, el procedimiento es increíblemente directo.

Básicamente, se empieza con todos los pesos en 0. Después se intenta el perceptrón con todas las muestras, una a la vez. Siempre que el perceptrón cometa un error, se cambian los pesos de modo que el error se haga menos probable, en cualquier otro caso, no se hace nada.

- Por el momento suponga que el perceptrón dice no, al producir un 0, cuando debería decir sí, produciendo un 1. Se puede aumentar la probabilidad de que diga sí la próxima vez si se incrementan los pesos asignados a aquellas cajas lógicas que producen 1 en el instante en que se comete el error. Una forma de asegurar el posible descubrimiento de un conjunto completamente eficaz de pesos consiste en aumentar cada uno de tales pesos en 1.
- De manera simétrica, siempre que el perceptrón dice sí cuando debería decir no, se disminuyen los pesos asignados a las cajas lógicas que producen 1, con la disminución igual a 1.
- En ningún caso se alteran los pesos asignados a las cajas que producen 0. Después de todo, tales pesos están multiplicados por 0, de modo que no es posible que al jugar con ellos se pueda cambiar el resultado actual y sí podría hacerse erróneo algún otro resultado correcto.

Finalmente, para conseguir el efecto de un umbral entrenable, se añade una entrada virtual extra, cuyo valor siempre se supone es 1, a la caja lógica que acaba de pasar el valor de entrada. **Con esta añadidura, el perceptrón se puede ver como si tuviera un umbral de 0, y dice sí siempre que la suma ponderada de las salidas de la caja lógica sea mayor que 0.**

Observe que se puede hablar del procedimiento de entrenamiento del perceptrón de forma más concisa mediante el lenguaje de vectores. Las salidas de caja lógica y los pesos, expresados en notación vectorial, son $(l_1, l_2, \dots, l_n)$ y $(w_1, w_2, \dots, w_n)$. Sumar uno a cada uno de los pesos asignados a cajas lógicas que producen 1 es lo mismo que sumar el vector de salida de caja lógica al vector peso, según la siguiente descripción:

Para entrenar a un perceptrón:

Hasta que el perceptrón produzca el resultado correcto para cada muestra de entrenamiento, para cada muestra:

- Si el perceptrón produce una respuesta errónea:
 - Si el perceptrón dice no cuando debería decir sí, añada el vector de salida de caja lógica al vector peso.
 - En cualquier otro caso, reste el vector de salida de caja lógica del vector peso.
- En cualquier otro caso, no haga nada.

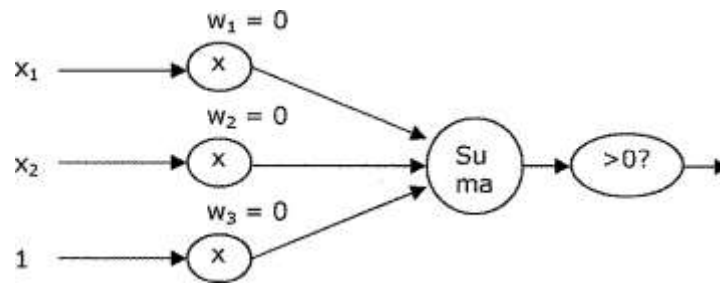


Figura 5.9

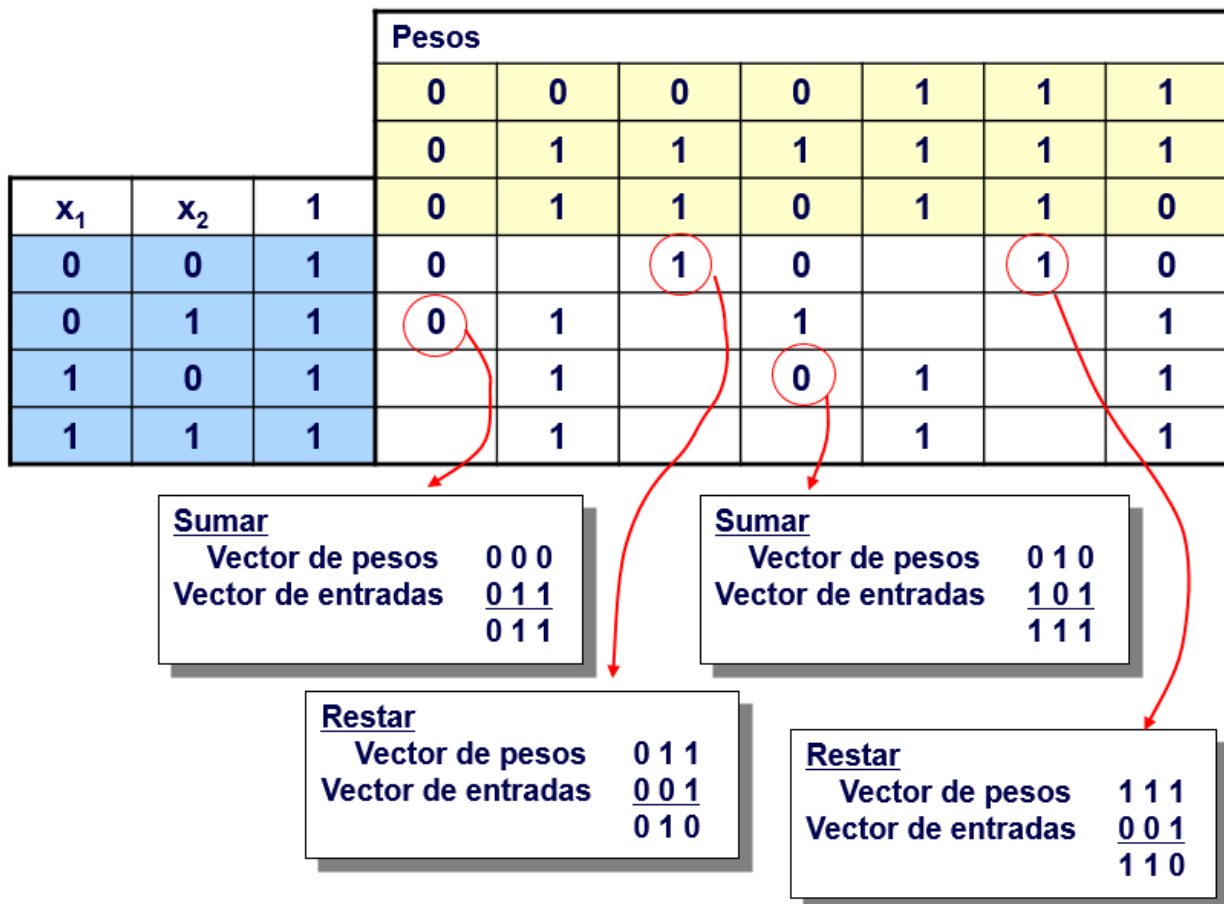
Por ejemplo, considere el perceptrón directo que se muestra en la Figura 5.9 y suponga que desea entrenarlo para que efectúe una función lógica OR, comenzando desde la línea de partida.

Como el perceptrón es directo, el vector de salida de caja lógica, (I_l, 12,13), es el mismo que el vector de entrada, (X_l). Por tanto, las muestras de entrada y las correspondientes salidas de caja lógica, para una OR lógica, son las siguientes:

Muestra	$x_1 = (I_1)$	$X_2 = (I_2)$	$X_3 = (I_3)=1$	Salida deseada
1	0	0	1	0
2	0	1	1	1
3	1	0	1	1
4	1	1	1	1

Procediendo en ciclos a través de estas muestras, finalmente el perceptrón aprende el OR lógico, después de cuatro cambios:

1. El primero ocurre después de un error en la segunda muestra durante el primer paso. En ese momento, el vector peso es (0 0 0), de modo que la salida es 0 cuando debería ser 1. En consecuencia, cuando el vector de entrada (0 1 1), se suma al vector peso, el nuevo vector peso es (0 1 1). Las siguientes dos muestras encontradas durante el primer paso producen 1, como deberían, de modo que no hay que hacer más cambios durante el primer paso,
2. Durante el segundo paso, la primera muestra produce un 1, pero debería producir un 0. En consecuencia, el vector de entrada (0 0 1) se resta del vector peso (0 1 1), lo que produce un nuevo vector peso (0 1 0).
3. Sin embargo, con este cambio, la tercera muestra produce un 0 cuando debería producir un 1. Por tanto, el vector de entrada, (1 0 1), se suma al vector peso, (0 1 0), lo que produce un nuevo vector peso, (1 1 1).
4. A continuación, la primera muestra produce de nuevo un error durante el tercer paso. El resultado debe ser 0, pero es 1. Al restar el vector de entrada, (0 0 1), del vector peso, (1 1 1), se produce un nuevo vector peso, (1 1 0), que posteriormente funciona para todas las muestras.



12.2 La separación lineal y el problema del XOR

El **teorema de convergencia del perceptrón** de Rosenblatt (1962), garantiza que el perceptrón encontrará un estado solución, es decir, aprenderá a clasificar cualquier conjunto de entradas linealmente separables.

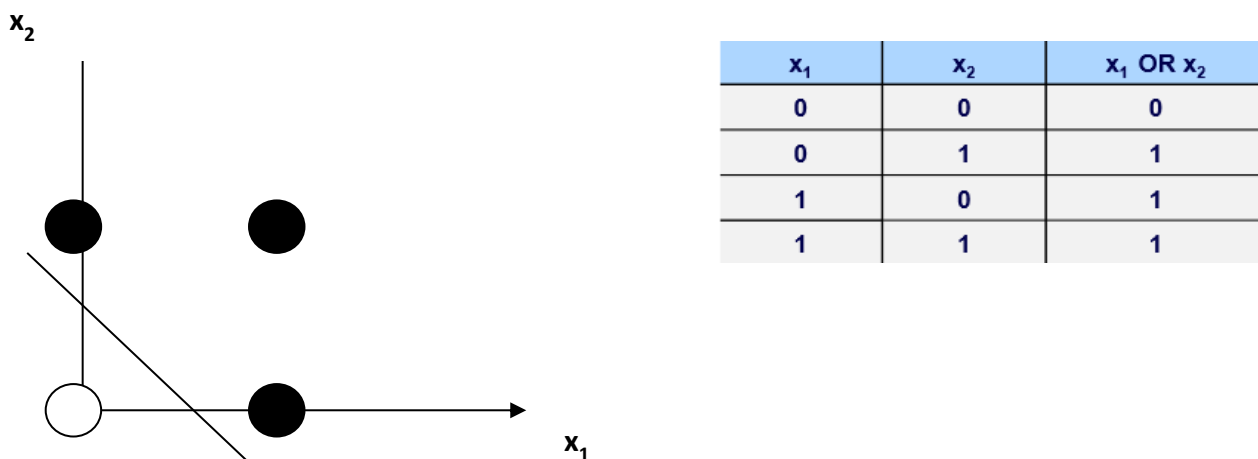


Figura 5.10

La introducción de los perceptrones a finales de la década de los cincuenta causó un gran entusiasmo. Suponía un dispositivo que se asemejaba mucho a una neurona y para el que había disponibles algoritmos de aprendizaje bien definidos. Hubo mucha especulación acerca de cómo podrían formarse sistemas inteligentes a partir de bloques contruidos con perceptrones.

En su libro Perceptrones, Minsky y Papert (1969) pusieron fin a dicha especulación analizando las capacidades de cálculo de dicho sistema. Ellos advirtieron que mientras el teorema de convergencia

garantizaba una correcta clasificación de la información linealmente independiente, la mayoría de los problemas no proporcionaban este tipo de datos tan bellos.

En realidad, el perceptrón es incapaz de aprender a resolver algunos problemas verdaderamente sencillos. Un ejemplo dado por Minsky y Papert es el problema OR-Exclusiva (XOR):

Dadas dos entradas binarias, dará salida 1 si solamente una de las entradas está conectada, y dará salida 0 si ocurre de modo contrario. Se puede considerar a XOR como un problema de clasificación de patrones en el que existen 4 patrones y 2 salidas posibles (ver Figura 5.11).

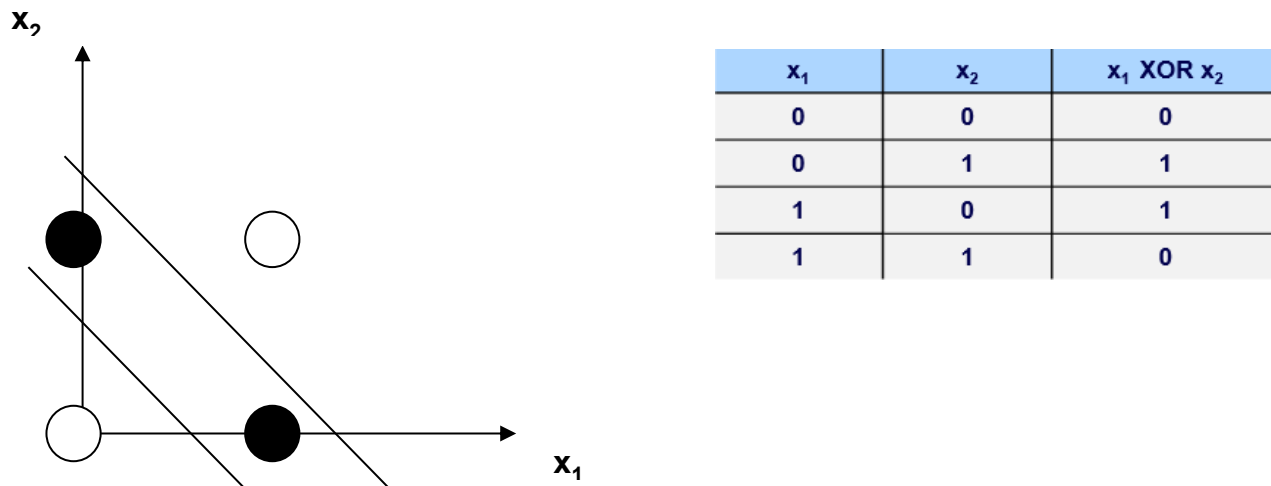


Figura 5.11

El perceptrón no puede aprender a crear una superficie lineal de decisión que separe estas diferentes salidas, porque dicha superficie de decisión no existe. **No hay una única recta que pueda separar las salidas 1 de las salidas 0.** Minsky y Papert proporcionaron varios problemas con estas propiedades.

Démonos cuenta que aquí la deficiencia no está en el algoritmo de aprendizaje del perceptrón, sino en el modo en el que el perceptrón representa el conocimiento.

- Si se pudiera dibujar una superficie de decisión elíptica se podrían englobar las dos salidas "1" en el espacio XOR. Sin embargo, los perceptrones son incapaces de modelar dichas superficies.
- Otra idea puede ser la de emplear dos escenarios diferentes para el dibujo de líneas. Se podría dibujar una línea para aislar el punto ($x_1 = 1$, $x_2 = 1$) y después otra línea para dividir los tres puntos que quedan en dos categorías. Utilizando esta idea se podría construir un perceptrón "multicapa" para solucionar el problema.

13. Redes de Hopfield

La historia de la IA es curiosa. Los primeros problemas con los que se enfrentaron los investigadores de la IA fueron problemas como el ajedrez o la demostración de teoremas, porque pensaban que en la solución de estos problemas se hallaba la esencia de la inteligencia.

La visión y la comprensión del lenguaje (tareas que un niño de cinco años ya realiza correctamente) no eran considerados problemas difíciles. Hoy en día ya se tienen programas expertos de ajedrez, así como programas expertos en realizar diagnósticos médicos, pero ningún programa puede llevar a cabo las características básicas de percepción que tiene un niño.

Los investigadores de redes neuronales reconocen que hay una falta de coincidencia evidente entre la tecnología empleada en el procesamiento de la información en una computadora estándar y la tecnología usada por el cerebro.

Además de estas tareas de percepción, la IA está empezando a tratar ahora los principales problemas referentes a la memoria y al razonamiento de sentido común. Ya es conocida la falta de sentido común de las computadoras. Mucha gente cree que el sentido común es una consecuencia del almacenamiento masivo de conocimiento, y aún más importante, de nuestra capacidad de acceder a conocimientos relevantes rápidamente, sin esfuerzos y a su debido tiempo.

Por ejemplo, cuando se lee la descripción "gris, grande, mamífero" automáticamente se piensa en elefantes y en sus características asociadas.

Accedemos a la memoria mediante contenidos.

En las implementaciones tradicionales, el acceso por contenido da lugar a complicados procedimientos de búsqueda y emparejamiento. Las redes masivamente paralelas sugieren un método más eficaz.

Hopfield (1982), introdujo una red neuronal que propuso como una nueva teoría de la memoria. Una red de Hopfield tiene las siguientes importantes características:

- Representación distribuida: una memoria se almacena como un patrón de activación a través de un conjunto de elementos de proceso. Las memorias pueden estar superpuestas una sobre otra. Las diferentes memorias se representan por diferentes patrones sobre el mismo conjunto de elementos de proceso.
- Control asíncrono y distribuido: cada elemento de proceso toma decisiones basadas únicamente en su propia situación local. Todas estas situaciones locales se unen para alcanzar una solución global.
- Memoria direccionable por contenido: se puede almacenar un determinado número de patrones en una red. Para recuperar un patrón únicamente se necesita una parte específica de él. La red encuentra automáticamente el emparejamiento más próximo.
- Tolerancia a fallos: aunque algunos de los elementos procesadores de la red fallasen, ésta todavía funcionará adecuadamente.

¿Cómo se alcanzan estas características?

En la Figura 5.12 se muestra una sencilla red de Hopfield. Los elementos de proceso, también llamados unidades, siempre se encuentran en uno de dos posibles estados, activos o inactivos.

En la figura **las unidades en negro están activas, y las unidades en blanco están inactivas**. Las unidades están conectadas unas con otras por conexiones simétricas y con pesos.

Una **conexión con peso positivo** indica que las dos unidades tienden a activarse la una a la otra. Una **conexión con peso negativo** permite que una unidad activa desactive a su unidad vecina.

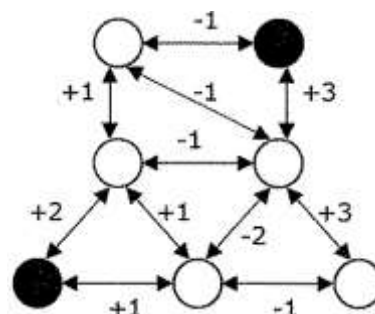


Figura 5.12

La red funciona del siguiente modo:

Se elige una unidad aleatoriamente. Si alguna de sus vecinas está activada, la unidad calcula la suma de los pesos en las conexiones de esas unidades. Si la suma es positiva, la unidad se activa y si ocurre de modo contrario, se desactiva.

Entonces se elige otra unidad aleatoriamente y se repite el proceso hasta que la red alcanza un **estado estable**, es decir, hasta que no quede ninguna unidad que pueda cambiar de estado. Este proceso se denomina **relajación paralela**.

Si la red comienza a trabajar en el estado que se muestra en la Figura 5.12, la unidad de la esquina inferior izquierda tenderá a activar la unidad que se encuentra por encima de ella. Esta unidad, por otro lado, tenderá a activar la unidad que se encuentra por encima de ella, pero la conexión inhibidora que procede de la unidad superior derecha aborta este intento, y así sucesivamente.

Esta red tiene únicamente cuatro estados estables distintos, que son los que se muestran en la Figura 5.13. Dado un estado inicial, la red necesariamente se asentará en una de estas cuatro configuraciones. La red puede verse como un "almacenador" de los patrones de la Figura 5.13.

La mayor contribución de las redes de Hopfield es la de mostrar que dado un conjunto de pesos y un estado inicial, su algoritmo de relajación paralela en algún momento llevará a la red hacia un estado estable. No puede no existir divergencia u oscilación.

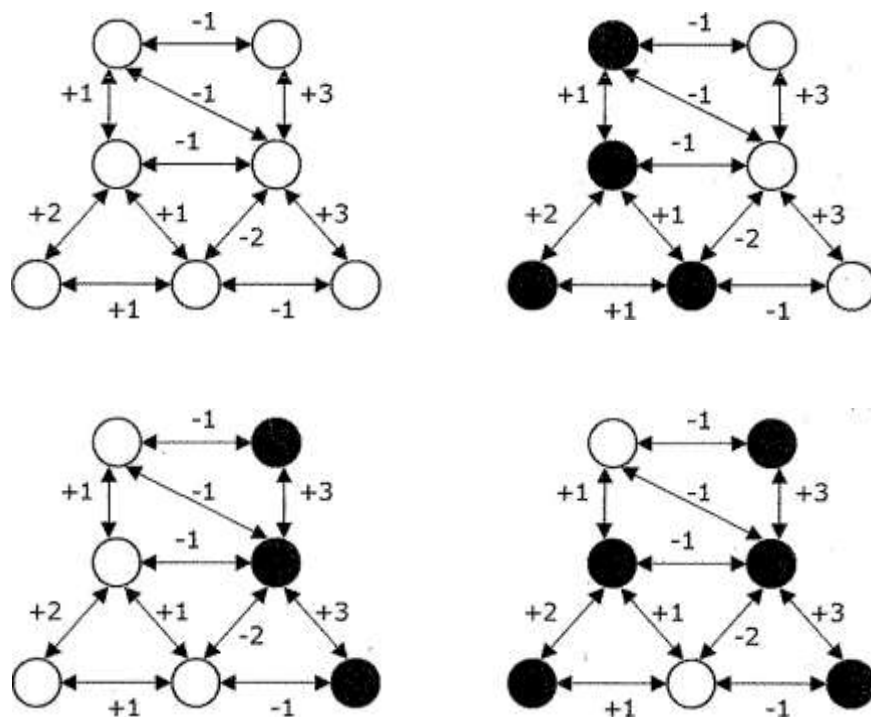


Figura 5.13

La red se puede utilizar como una memoria direccionable por contenido haciendo que las actividades de las unidades se correspondan con un patrón parcial. Para recuperar un patrón, únicamente se necesita una parte de él. Entonces la red se asentará en el estado estable que mejor se empareje con el patrón parcial. En la Figura 5.14 se muestra un ejemplo de lo anterior.

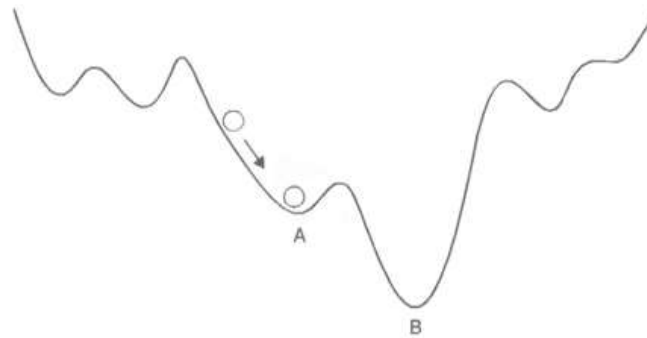


Figura 5.14

La relajación paralela no consiste más que en una búsqueda.

Resulta útil pensar en los diferentes estados de una red como formando un espacio de búsqueda como se muestra en la Figura 5.15.

Un estado que se haya elegido aleatoriamente se transformará a sí mismo en uno del tipo de mínimo local que es el estado estable más cercano. Esta es la forma de obtener un comportamiento direccionable por contenido.

También tenemos un comportamiento corrector de errores. Supóngase que se tiene la descripción "gris, grande, pez, que come plancton". Automáticamente imaginaremos una ballena, aún sabiendo que ésta es un mamífero y no un pez. Por tanto, aún cuando en el estado inicial existan inconsistencias, la red de Hopfield se asentará en la solución que viole el menor número posible de restricciones que ofrecen las entradas. Los procedimientos tradicionales de emparejamiento-y-recuperación son menos prohibitivos.

Ahora supóngase que una unidad falla de repente haciéndose activa o inactiva cuando no debe. Esto no representaría mayor problema, ya que las unidades que le rodean rápidamente volverían a ponerla en el camino correcto.

Sería necesario el esfuerzo de muchas unidades erróneas para hacer que la red se saliera de un estado estable. En las redes compuestas por miles de unidades altamente interconectadas, dicha tolerancia al fallo es todavía más patente, ya que las unidades y las conexiones pueden desaparecer completamente sin afectar de un modo adverso el comportamiento general de la red.

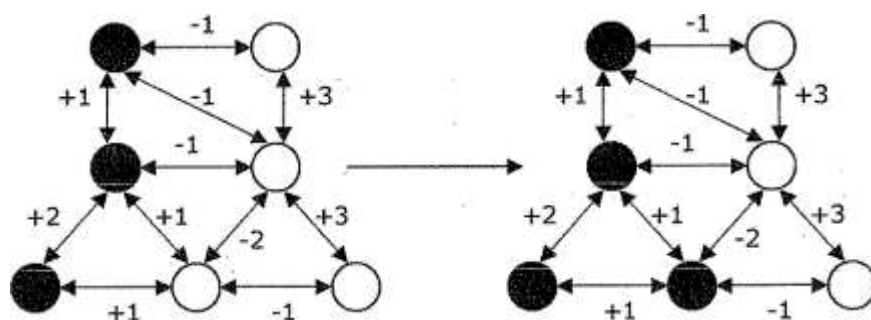


Figura 5.15

14. Máquinas de Boltzman

Una máquina de Boltzmann es una variación de una red de Hopfield.

Recuerde que en una red de Hopfield las unidades se conectan entre sí con pesos simétricos. Las unidades actualizan sus valores de forma asíncrona observando sus conexiones locales con otras unidades.

Las redes de Hopfield, además de servir como memorias direccionables por contenido, pueden resolver varios problemas de verificación de restricciones. La idea es contemplar cada unidad como una "hipótesis", y situar pesos positivos entre las unidades que representen hipótesis compatibles o que se apoyan mutuamente, y pesos negativos en las conexiones entre unidades incompatibles. Como una red de Hopfield converge hacia un estado estable, intenta asignar la certeza o falsedad de las distintas hipótesis al violar el menor número de restricciones posibles.

El principal problema de las redes de Hopfield es que convergen hacia mínimos locales.

Tener muchos mínimos locales es adecuado para construir una memoria direccionable por contenido, pero para las tareas de verificación de restricciones es necesario encontrar el estado globalmente óptimo de la red. Este estado se corresponde con una interpretación que satisfaga tantas restricciones como sea posible.

Desafortunadamente, las redes de Hopfield no pueden encontrar soluciones globales porque se sitúan en los estados estables por medio de un algoritmo completamente distribuido.

Si una red alcanza un estado estable como el de la Figura 5.15, ninguna unidad querrá cambiar su estado para moverse hacia arriba. Así, la red nunca alcanzará el estado óptimo B. Si varias unidades deciden cambiar su estado simultáneamente, la red podría escalar la subida y descansar sobre el estado B. Es necesaria una forma de situar las redes en los estados globalmente óptimos si mantenemos nuestro enfoque de algoritmo distribuido.

A la vez que se desarrollaron las redes de Hopfield, apareció en la literatura una nueva técnica de búsqueda denominada **enfriamiento simulado**.

El enfriamiento simulado, es una técnica para encontrar soluciones globalmente óptimas en problemas combinatorios.

Hinton y Sejnowsky (1986) combinaron las redes de Hopfield y el enfriamiento simulado para producir redes denominadas **máquinas de Boltzmann**.

Para entender cómo se aplica el enfriamiento, vuelva a la Figura 5.15 e imagínela como una caja negra.

Imagine una bola rodando por la caja. Si no puede verse dentro de la caja negra, ¿cómo podríamos convencer a la bola para que se vaya al valle más profundo? Sacudiendo la caja, por supuesto.

- Si se sacude demasiado violentamente, la bola podrá saltar de un valle a otro aleatorio. Esto es, si la bola estaba en el valle A, podría saltar al valle B; pero si la bola estaba en el valle B, ésta podría saltar al valle A.
- Sin embargo, si se sacude con demasiada suavidad, la bola podría saltar dentro del valle A, pero nunca saltar a otro valle.
- La solución que sugiere el enfriamiento es sacudir la caja violentamente al principio, y gradualmente ir haciéndolo más suavemente. En algún punto, la probabilidad de saltar de A a B es mayor que la de saltar de B a A. Será muy probable que de esta forma la bola se encuentre en el valle B, y como cada vez la sacudida es más suave, será incapaz de escapar de él. Esto es justo lo que queremos.

¿Cómo se implementa esta idea en una red neuronal?

Las unidades de las máquinas de Boltzmann actualizan sus estados binarios individuales mediante una regla estocástica, en lugar de una determinista.

La probabilidad de que una determinada unidad se active viene dada por p :

$$p = \frac{1}{1 + e^{-\Delta E / T}}$$

donde ΔE es la suma de las líneas de entrada activas de las unidades, y T es la "temperatura" de la red.

La actualización estocástica de las redes es muy similar a la actualización de las redes de Hopfield, excepto en el factor que introduce la temperatura.

A altas temperaturas, las unidades presentan un comportamiento aleatorio, mientras que, a temperaturas bajas, las unidades se comportan como redes de Hopfield.

El enfriamiento es el proceso de ir gradualmente de altas a bajas temperaturas.

La aleatoriedad que añade la temperatura ayuda a la red a escapar de los mínimos locales.

Se trata de un procedimiento de aprendizaje para las máquinas de Boltzmann, es decir, un procedimiento que asigna pasos a las conexiones entre las unidades dando un conjunto de estados iniciales y finales de entrenamiento.

Si el enfriamiento se produce de forma adecuada, las máquinas de Boltzmann podrían evitar los mínimos locales y aprender a calcular cualquier función calculable de entradas y salidas de tamaño fijo.